MEDHAVA

Medhava — the Architect

What this system is, and why it is shaped this way. 22 modules · 113 apps · 151 tables · 285 rules · 19 layers with 57 ways out.

Every figure on this page is counted from the source files at the moment it was written. None was typed from memory, which is why they have already changed twice.

How to read this, and its companion

Document	Answers	Read it when
MEDHAVA_ARCHITECT (this one)	WHAT and WHY	You are deciding whether this design is right, or you need to argue with a decision
MEDHAVA_BUILD_GUIDE	HOW, in order	You are building it, from an empty machine to a deployed product

The test for which document a sentence belongs in: does it survive a change of language, framework or host? *"Money is whole paise, never a float"* survives, so it is here. *"Run npm ci"* does not, so it is there.

Nothing in this document claims to be running. It is a design, and the point of writing it down is so that what gets built is the thing that was decided rather than the thing that was convenient on the day.

Part 1 · One system, many businesses, none of them alike

Medhava is one piece of software that many separate businesses run their whole operation on. Each sees only its own information. Each sees it in its own words. None of them has a copy of the code.

That last sentence is the entire design problem, and everything else in this document follows from it. The moment one customer gets a branch in the code — a special case, a fork, a "we will add a setting just for you" — the software has started to split, and in two years there are as many versions as there are customers, each needing its own fix for the same bug.

So the things that differ between businesses are DATA, not code. Their words, their steps, their extra fields, their rules, their rates, their people, how many companies they run and how many places they sell. A steel plant, a clothing manufacturer, a clinic, a law practice and a single creator selling courses all run the same build. What differs is what is in their rows.

1.1 · A business is a row, not a deployment

> **platform** — *One piece of software that many separate businesses use at the same time, each seeing only its own information.* *Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.* **tenant** — *One business using the platform. Its people, its data and its settings are its own.* *Us building mein ek office. Aapka office, aapka saamaan, aapka taala.* **database** — *Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.* *Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.* **row** — *One single record — one customer, one order, one payment.* *Register mein ek line. Ek line matlab ek entry.*

What. Every customer of Medhava is a **tenant** — one row in one table, on the same database as every other tenant. Not a copy of the software, not a separate server, not a schema of their own.

Why. The alternative is a deployment per customer, and it is a trap that looks like safety. Fifty customers becomes fifty upgrades, fifty backups, fifty places a security fix has to land, and the first time one of them lags a version the support answer becomes "which build are you on". One row means one upgrade for everybody and one place to look when something is wrong.

What would make this the wrong decision. A customer needs data physically separate for a regulator, or one customer is so large its load hurts the others. Both are real, and both are answered the same way — that tenant moves to its own database with the SAME schema and the SAME code. The design survives; only the hosting changes. What would break the design is a customer needing different LOGIC.

1.2 · Isolation is the database’s job, never a WHERE clause somebody remembers

> **row-level security** — *A lock inside the database itself, so one business physically cannot read another business’s records — even if the software above it has a bug.* *Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.* **table** — *One kind of information inside the database — all your customers in one, all your orders in another.* *Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.*

What. Every business table carries `company_id`. Row-level security policies are attached in the database itself, so a query that forgets to filter returns nothing rather than everything. The same mechanism one level up keeps two TENANTS apart, on `tenant_id`.

Why. A filter in a screen can be removed by anybody editing that screen. A policy in the database cannot be removed by editing a screen. The application checks the same thing again at its own layer — two independent layers, because one layer is one mistake away from a customer reading another customer’s orders.

Three details decide whether this works at all, and all three are easy to get wrong: `FORCE ROW LEVEL SECURITY` so the table's owner is subject to its own policy; the application connecting as a role that is **neither superuser nor table owner**, because Postgres bypasses RLS for superusers and FORCE does not stop them; and an explicit guard so an UNSET company setting refuses rather than matching everything. The guard covers a company set to the empty string; one that was never set is NULL and the ::uuid cast raises instead. Both refuse, and knowing which is the difference between a guarantee and a lucky accident.

What would make this the wrong decision. Nothing here is wrong-if. This is the one decision in the document with no acceptable alternative: a cross-tenant leak is not a defect, it is an incident, and it is the single highest-risk item in the whole design.

1.3 · The count of anything a business owns is a row count

> **schema** — *The written plan of what information the system keeps and how the pieces connect.* *Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.*

What. How many companies, how many channels, how many modules are switched on, how many people — none of these numbers appears in the code. They are counted from rows.

Why. This tenant runs three companies and sells on seven channels today. Both numbers have already changed once during the writing of this system, and the owner's own words about the second were "marketplace can be 6 or 7 or 10, why are you holding it so strong". He is right, and a number in the code would make him wrong.

The proof is not a promise: the core test posts across a **10 x 10 grid** of companies and channels, then runs 11 x 11 with no code changed, and `brand/site/checkstatic.js` fails the build if a business count is ever compiled in.

What would make this the wrong decision. Never. A count in the code is always a bug waiting for a customer to grow.

Part 2 · Every value has a date, and the past does not move

A salary is not a number. It is a number **that was true between two dates**, and the difference decides whether last year's books still add up after somebody gets a raise.

This is the idea that most business software gets wrong, and it is worth being blunt about the failure mode: a system that stores "salary = 20,000" and lets somebody edit it has just changed what every past month costs. The payroll for last April silently becomes a different number. Nobody notices until an auditor asks.

2.1 · Effective-dated and append-only, everywhere a value can change

> **effective date** — *The date a change starts applying from. Records made before it keep the old value; records after it use the new one.* *Naya rate 1 tarikh se lagu. Purane mahine ka bill purane rate se hi banega — woh apne aap nahin badlega.* **audit trail** — *An automatic record of every change — what changed, who*

> *changed it, and when.* *Har entry ke saath naam aur time apne aap likha jaata hai. Baad mein koi bole "maine nahin kiya", toh register bata deta hai.*

What. A change never overwrites. The open row is closed at the day before the change, and a new row is appended carrying the date it starts from and who made it. Asking for a value "as of" any date returns what applied then.

Why. Three properties fall out of it and all three matter:

Future-dated entries activate themselves. A raise recorded today for next month is simply the answer when next month is processed. Nobody has to remember.

A closed month does not move. Renaming something today cannot change what last year cost.

No match is an error, never zero. Asking for a value before anything ever set it raises rather than quietly returning nothing — because a silent zero is how a wrong number reaches a real person’s payslip, and a raise is a question somebody answers in a minute.

What would make this the wrong decision. A value genuinely has no history and never will — a colour name, a country code. Dating those adds ceremony for nothing. The test is whether anybody would ever ask "what was it in March".

2.2 · A person is a series of spells, not a join date and a leave date

What. Employment is a list of periods with gaps allowed. Somebody can work, leave, and come back on a new spell with their history intact and nothing about the old spell rewritten.

Why. Because that is what actually happens. This tenant has, right now, five different states in play and they are not interchangeable:

somebody **working** · somebody **on leave** for a month, still on the roster · somebody **inactive who can return** — the owner’s words about one contract worker are "we can associate in future" and about two others "they can come to work as contract basis whenever I need" · somebody who has **left** · and a **trial**, who came for a few days, was paid, and went.

Collapsing those into one "active" flag loses the difference between a person on leave and a person gone, which is the difference between a payslip and no payslip.

What would make this the wrong decision. Never, in any business that employs people. The moment a model cannot express "came back", somebody starts keeping the truth in a spreadsheet beside the system.

2.3 · A trial has no employment record at all, and is still paid

What. Somebody can be paid for days worked without ever being onboarded — no spell, no salary history, no threshold. The **payment is the record**.

Why. The owner’s description: "can come today and leave tomorrow if we didn’t like them or negotiation failed". A system that requires a person to exist before attendance or a payment can be entered cannot represent that day, so the day gets entered wrongly or not at all — and either way the wage is missing from the books.

Nothing is derived for a trial, so nothing raises "salary missing" — there is no salary to miss. The cost still lands in the right company and the right month. If the trial works out, they become a regular person with a start date and the trial days stay in history as trial days.

What would make this the wrong decision. A business that legally cannot pay anybody without a contract on file. Then the contract becomes the precondition — but that is a RULE the tenant switches on, not a shape the software forces on everybody.

Part 3 · Money, and the two mistakes that are hard to reverse

Two decisions about money have to be right before the first invoice is posted, because both are effectively impossible to fix afterwards on live data.

3.1 · Money is whole paise in an integer. Never a float.

> **integer paise** — *Money stored as a whole number of paise instead of a decimal, so amounts are exact and rounding can never quietly lose a rupee.* *Paisa hamesha poore paise mein ginte hain, aadha-adhoora kabhi nahin — isliye hisaab kabhi ek rupya idhar-udhar nahin hota.*

What. Every money column is an integer number of paise and its name says so — `_paise`. No money value is ever a floating-point type, and a gate reads both schemas to prove it.

Why. The arithmetic is not approximately right, it is wrong in a way that accumulates. In a real database, `0.1 + 0.2` in floating point is `0.30000000000000004` — measured, not quoted from a blog. Across a hundred thousand invoice lines that becomes a reconciliation nobody can close, and the errors are unevenly distributed so they do not cancel out.

The naming half matters as much as the type: `total numeric` reads as rupees to one developer and paise to the next, and the difference is a factor of a hundred in the books.

What would make this the wrong decision. Never. There is no version of this where floats are acceptable for money.

3.2 · Double-entry underneath, whatever the screen looks like

What. Every financial event produces balanced journal entries. Screens can look like anything; the ledger underneath is the ledger.

Why. Because the alternative — a "transactions" table with a sign column — cannot answer "why does this balance not match" without a human reading rows. Double entry answers it structurally: if the two sides disagree, the entry was refused when it was written, not discovered at year end.

What would make this the wrong decision. A business that never needs a balance sheet. Almost none, once they have a lender, an auditor or a tax authority.

3.3 · A missing rate posts nothing and says so

What. Where a value needed for a calculation was never supplied, the calculation reports **Unresolvable** and pays zero, flagged. It never guesses, never carries a neighbour's figure forward, and never silently posts zero as though zero were the answer.

Why. This rule exists because it was broken. One contract worker's file carried ₹100/hour copied from a different contract worker, because that figure was to hand and the real one had never been stated. It looked completely reasonable and it was fiction, and fiction on a payslip is the worst kind. The rate was removed and every remaining rate now has to cite the line that states it.

Two people in this tenant are on piece rate with no rate stated. Their months raise. That is the system working.

What would make this the wrong decision. Never. "Approximately paid" is not a thing.

Part 4 · Modules, apps and the cascade between them

The system is organised as modules. A module is one area of work — sales, purchase, staff, accounts — and holds a set of screens that belong together. Each module declares what it READS and what it WRITES, and those declarations are what wire the system together.

4.1 · Modules declare their reads and writes, and the wiring is derived from that

> **module** — *One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.* *Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.*

What. No module calls another module directly. Each says what it produces and what it consumes; the connections between them are generated from those declarations, and the module dependency map in the documents is drawn from the same source rather than maintained by hand.

Why. Hand-drawn architecture diagrams are wrong within a month of being drawn, and nobody finds out because nothing checks them. A derived one cannot drift: if a module starts writing something new, the map changes on the next build. It also means a module can be switched OFF for a tenant who does not need it, and what breaks is knowable in advance.

What would make this the wrong decision. A module needs to call another one synchronously and wait — a real case for tight, latency-sensitive work. Then it is a direct call, declared as such, and the exception is visible rather than being one more undocumented arrow.

4.2 · A rulebook, where every rule says what the system will NEVER do instead

What. Every module carries numbered rules. Each states what happens **and** what the system refuses to do in its place. A rule marked ENFORCED must name a file and a test that really exist, and the build fails otherwise.

Why. "The system validates input" is not a rule, it is a mood. "A stock movement quantity must be positive; a zero or negative quantity is refused, never absorbed as a correction" is a rule — it tells you what somebody was

tempted to do instead and why that was rejected.

The `never` half is what makes a rulebook checkable. A rule without one is a description, and the checker rejects it as such.

What would make this the wrong decision. Never — but the risk is real: a rulebook can rot into 285 sentences nobody reads. The defence is that the rules are injected into the documents from one source and the ENFORCED ones have to point at a passing test.

4.3 · Configuration is a pack, and what a business changed after is an overlay

> **industry pack** — *A settings file that teaches the system your trade — what you call things, the stages your work moves through, the documents you issue.* *Ek hi machine, alag-alag saancha. Saancha badal do, wahi machine doosri cheez banane lagti hai.*

What. A trade starts from a **pack** — the words, stages, fields and modules that suit that industry. Everything the business changes afterwards is an **overlay**: effective-dated, append-only, and resolved as of any date.

Why. Two different lifetimes. A pack is where a trade STARTS and is versioned with the software. An overlay is what one business did on a Tuesday, and it must be changeable by that business without anyone touching the repository.

Before the overlay existed, six things the documents promised a tenant could change — its vocabulary, stages, fields, documents, which rules are on and which modules are on — were all files in the source tree, which meant every one of them was a code deployment by the vendor. A promise a document makes and the code cannot keep is the beginning of a fabrication.

What would make this the wrong decision. A tenant needs something no pack or overlay can express. That is a real signal — it means the thing they need is genuinely code, and the right answer is a new capability for everybody, not a branch for them.

Part 5 · The stack, and the fact that none of it is load-bearing

Nineteen layers, each with a default choice, at least two named alternatives, and the interface that makes swapping one possible. The register is generated from a single source and a gate refuses a layer that names fewer than two ways out.

5.1 · Every layer names its alternatives before it is chosen

> **interface** — *A written promise about what a part of the system does, without saying which product does it — so the product underneath can be swapped without anything above noticing.* *Bijli ka socket. Socket ka size fix hai; usme koi bhi company ka plug lag jaata hai.* **adapter** — *A small piece of code that translates between the system and one outside service, so the rest of the system never has to know which service is in use.* *Travel adapter. Andar ka appliance wahi, bas plug ko us desk ke socket ke hisaab se badal diya.*

What. A layer is not allowed into the design without naming what could replace it and what the interface between it and everything else is.

Why. Not because the alternatives will be used — most never are — but because being forced to name them is what proves the layer was chosen rather than defaulted to. A layer with no alternative is a layer nobody thought about, and it is the one that becomes impossible to move three years later when its vendor triples the price.

What would make this the wrong decision. The alternatives listed are not real. A named alternative nobody has checked is worse than an honest "this one is load-bearing", because it manufactures confidence.

5.2 · Free first, and a paid tool must name its free option and its trigger

> **provider** — *A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery.* *Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.*
> **fallback** — *The next option the system automatically moves to when the first one fails or is unavailable.* *Bijli gayi toh inverter. Inverter gaya toh mombatti. Andhera kabhi nahin hota.*

What. Every capability starts on something free. Where a paid tool is chosen, the register must record what the free option was and the specific trigger that justifies paying.

Why. A small manufacturer's software budget is real money. "We use the paid one because it is better" is not a decision, it is a preference. "We use the paid one once outbound messages pass X a month, because below that the free tier covers it" is a decision somebody can check against their own numbers.

What would make this the wrong decision. A free option does not exist for a capability. Then the register says so plainly rather than inventing one.

Part 6 · The data model

One schema, in build-phase order, so the first phase can be run without reading the rest. Every business table carries `company_id`, row-level security, FORCE, and a grant — all four, because three of them without the grant is a table nobody can read, and three without the policy is a table everybody can.

6.1 · The schema is executed by a test, not read by one

> **migration** — *A recorded change to the shape of the database, so every copy of the system can be updated the same way, in the same order.* *Naksha badla toh likh ke rakha — taaki har site pe wahi badlav, usi tarike se ho.*

What. The production schema is loaded into a real PostgreSQL in the test suite and the isolation is asked of the database rather than asserted about the file.

Why. Because a text check is structurally incapable of finding the thing that was actually wrong. The committed schema passed every text assertion for months and had **never been executed** — the first time anything ran it, it

failed on its very first policy with `role "authenticated" does not exist`, and because the file is one transaction, nothing was created at all.

The test also opens with a negative control that deliberately leaks and REQUIRES the leak: if the harness cannot detect a cross-company read, every check after it is decoration and the file aborts rather than reporting a pass.

What would make this the wrong decision. Never. "Proved by a test that tries" is the standard, and a schema nothing runs is a document.

6.2 · Delete nothing — soft-delete, or be an append-only log

What. Every table that holds a business record can record that it was voided. Tables that are genuinely event logs do not need it, and each one says in its own words why it is an event rather than a record.

Why. Because "who deleted the March invoice" has to have an answer. The exemption list is the interesting part: a table added to it instead of given the column is the rule being waved through, which is why each exemption has to justify itself.

What would make this the wrong decision. Data a regulator requires to be truly erased. Then erasure is a deliberate, audited operation with its own record — not the ordinary delete path.

Part 7 · What the system refuses to do, on purpose

A design is defined as much by its refusals as its features. These are not limitations to be lifted later; they are the decisions that make the rest trustworthy.

7.1 · It never asks for a marketplace, bank or account password

> **encryption** — *Scrambling information so that even somebody who steals the file cannot read it.* *Apni hi code-bhasha mein likhna. Chori bhi ho jaaye toh padha nahin jaata.*

What. Not at onboarding, not for a migration, not "just this once" for support. Connections are made with keys the platform is granted, which the tenant can revoke.

Why. A password gives away everything the account can do, forever, to anybody who later reads the place it was stored. A key can be scoped and revoked. This is written into the product's own promise, and the code and the conversation both have to honour it — including refusing a password that somebody volunteers.

What would make this the wrong decision. Never, and the pressure to bend it is real: a marketplace with no API, a migration that would be quicker by logging in as the customer. The answer to both is that the work is done a slower way or not at all. A promise with an exception is not a promise.

7.2 · A raw API key is never stored, and there is nowhere to store one

> **permission** — *One specific thing a role is allowed to do, like approving a discount or viewing salaries.* *Guchhe ki ek chaabi. Ek chaabi ek darwaza.*

What. Keys are shown once at issuance. What is kept is a hash and a scope. The table has **no column** a raw key could be written to.

Why. A "we never log credentials" flag beside a raw-key column is a promise the schema itself breaks. The way a schema keeps that promise is by having nowhere to break it, and a test asserts the absence of such a column rather than the presence of a flag.

What would make this the wrong decision. Never. If a key genuinely must be replayable — some payment gateways ask for it — that is a secret store with its own access log and its own rules, not a column on a business table that every report can read.

7.3 · A person's name never appears in logic

> **role** — *What a person is allowed to see and do — a manager sees more than a counter staff member.* *Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.*

What. No branch anywhere is taken because of who somebody is. Behaviour follows a flag the person carries — flat-salary, piece-rate, trial — and the flag is data.

Why. `if (staff === 'Karim')` works until Karim leaves, and then it silently applies to nobody while everybody assumes it still works. It also means the rule cannot be given to the next person without a developer. Two separate gates enforce this, one for each language.

What would make this the wrong decision. Never. The temptation appears whenever one person is genuinely an exception — and that is precisely when the exception should become a flag, because an exception worth coding is an exception worth naming.

Part 8 · The registers, derived

Four tables that are generated rather than maintained. Each is read from the one file that owns that fact, so none of them can drift from the software it describes.

The words the tables below introduce. The registers name every layer and module, which brings in vocabulary the argument above did not need.

> **backup** — *A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work.* *Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.* **backend** — *The part of the software you never see, which does the actual work — checks the rules, saves the records, calculates the totals.* *Hotel ka kitchen. Customer nahin dekhta, par khaana wahin banta hai.* **frontend** —

The part you see and click — the screens, the buttons, the forms. *Hotel ka dining hall aur menu card. Jo aapke saamne hai.* **API** *— The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.* *Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.* **storage** *— Where files are kept — photographs, invoices, scanned documents. Different from the database, which keeps information rather than files.* *Almari ke bagal wala godown. Register almari mein, par bade dabbe aur photo godown mein.* **queue** *— A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.* *Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.* **environment** *— A separate running copy of the system — one for trying things, one that customers actually use.* *Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.* **deployment** *— Putting a new version of the software in place so people start using it.* *Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.* **uptime** *— How much of the time the system is actually working and reachable.* *Dukaan mahine mein kitne din khuli rahi. Band rahi toh customer wapas chala gaya.* **model** *— The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question.* *Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.*

8.1 · The stack, and its ways out

Layer	What it does	Built on	Ways out
The database	Keeps every record — customers, orders, stock, vouchers — and answers questions about them.	**PostgreSQL**	3
File storage	Keeps photographs, invoices and scanned documents — the things too big to sit in the database.	**Any S3-compatible object store**	3
Cache and short-term memory	Holds recently used answers and sign-in sessions so common screens open instantly.	**Redis, or a Redis-compatible store**	3
The backend runtime	Runs the business rules, checks permissions, writes records and calculates totals.	**Node.js with TypeScript**	3
The API	The agreed way the screens, the mobile view and any outside system ask the backend for things.	**REST over HTTPS, with a written schema**	3
The frontend	Everything a person sees and clicks — screens, forms, tables, dashboards.	**React with TypeScript, screens generated from configuration**	3
Background work	Does the things nobody should have to wait for — sending a thousand messages, building a month-end report, pulling orders overnight.	**A queue backed by the database, with named workers**	3
Search	Finds a product, a customer or a document by a few typed letters, instantly.	**PostgreSQL full-text search**	3
Sign-in and permissions	Proves who somebody is, then decides what they are allowed to see and change.	**Sessions issued by the platform, with permissions checked in the backend and again in the database**	3
Keys and passwords the system uses	Holds the connection details and keys the software needs, away from the code.	**Environment variables on the server, readable only by the service account**	3
Messages to customers and staff	Sends WhatsApp messages, text messages and email — reminders, confirmations, statements.	**A message service with one adapter per provider, per tenant**	3
Storefronts and marketplaces	Brings orders in from a shop website or a marketplace, and sends stock and prices back out.	**A channel adapter per storefront or marketplace**	3
Taking payments	Collects money from customers online.	**A payment adapter per provider, with the card field hosted by the provider**	3
Delivery and couriers	Books a shipment, prints the label, and follows it to the door.	**A courier adapter per carrier**	3

Layer	What it does	Built on	Ways out
Artificial intelligence	Writes descriptions, tags photographs, summarises, and answers questions about your own data.	**A router in front of several providers, ending on one that needs nothing bought**	3
Where it runs	The machines that serve the website and the application.	**Containers on a virtual server**	3
Source control and automatic checks	Keeps the history of every change and runs every test before anything goes live.	**Git, with automatic checks on every change**	3
Watching it	Reports errors, measures speed, and tells you when something stops answering.	**Structured logs and error reporting, in an open format**	3
Making documents	Produces invoices, statements, labels and reports as files a person can print or send.	**HTML templates printed to PDF by a headless browser**	3

And what replaces each one. A count in a "ways out" column is a claim somebody checked; the sentences are what let a reader check it. Each is printed whole, because a named alternative with its reasoning removed is a name, and a name is not an escape route.

- **The database** — A managed Postgres service — same database, somebody else runs the machine · Postgres on your own server — the software is free, you supply the machine · MySQL or MariaDB, if a team already knows them well — the schema needs rework for the record-level locks
- **File storage** — A different S3-compatible provider — usually a URL and a key change · Files on your own server’s disk, with a backup copy elsewhere · A self-hosted object store such as MinIO, which speaks the same format
- **Cache and short-term memory** — Valkey — the open-source continuation of the same thing, same commands · Memory inside the application itself, which is enough until traffic grows · A database table, slower but with nothing extra to run
- **The backend runtime** — Any container host — the code is ordinary and carries no host-specific parts · Python or Go for a service that genuinely suits them, talking over the same API · A different Node framework — the business logic sits outside the framework on purpose
- **The API** — GraphQL for read-heavy screens, over the same underlying services · A direct connection for live screens that must update by themselves · Scheduled file exchange for partners who cannot call an API at all
- **The frontend** — Vue or Svelte — the screen definitions are plain data and do not care what draws them · Server-rendered pages where speed on a weak connection matters more than interaction · A native mobile shell reading the same screen definitions
- **Background work** — A Redis-backed queue when volume outgrows the database · A hosted queue service, behind the same interface · An external workflow tool such as n8n for steps a non-programmer should be able to edit
- **Search** — OpenSearch or Elasticsearch when catalogues grow large · Meilisearch or Typesense — small, fast, self-hostable · A hosted search service behind the same interface

- **Sign-in and permissions** — An identity provider for sign-in only, with permissions still decided here · A customer's own company sign-in, for enterprises that require it · Self-hosted Keycloak or Authentik, when nothing may leave the building
- **Keys and passwords the system uses** — A managed secrets service, when there are enough of them to be worth it · Self-hosted Vault or Infisical · Encrypted files kept outside source control
- **Messages to customers and staff** — Any WhatsApp provider — the adapter changes, the code that decides what to send does not · Text message and email as fallbacks when a message cannot be delivered · A shared inbox or an export, for a tenant with no messaging account at all
- **Storefronts and marketplaces** — A different storefront platform — a new adapter, and orders keep arriving · File import for a channel with no connection available · Manual entry, which must always remain possible
- **Taking payments** — Any other payment provider, behind the same interface · Bank transfer and UPI details recorded against the invoice · Cash on delivery, reconciled when the courier settles
- **Delivery and couriers** — A courier aggregator, which is itself just one more adapter · A different carrier directly · Manual booking with the tracking number typed in — always available
- **Artificial intelligence** — Any hosted model provider — an entry in the router, not a change to the system · A model running on your own machine, for work that is routine or private · Templates and rules with no model at all, which must always remain the last resort
- **Where it runs** — A managed container platform, when scaling by hand stops being fun · A different cloud, or a different country, for the same container · A machine in your own office, for data that must not leave it
- **Source control and automatic checks** — A different hosting service — a git repository moves with one command · Self-hosted Gitea or Forgejo · A separate build service reading the same repository
- **Watching it** — Any hosted error-tracking service · Self-hosted GlitchTip, or a Grafana and Prometheus stack · Log files plus an uptime checker, which is enough at the start
- **Making documents** — A dedicated PDF library for very high volume · A hosted document service · Spreadsheet or CSV output, which some readers prefer anyway

19 layers, 57 named alternatives between them. A layer is refused entry to this design without at least two, because a layer with no alternative is a layer nobody chose.

8.2 · The modules

#	Module	Apps	Rules
01	Platform	8	25
02	Design & Sampling	2	7
03	Inventory & Catalog	4	14
04	CRM	4	9
05	Sales	8	18
06	Planning & Requirements (MRP)	3	8
07	Purchase	3	12
08	Manufacturing	4	20
09	Quality & Compliance	2	7
10	Warehouse	3	8
11	Logistics	5	11
12	Accounting & GST	9	24
13	Treasury & Financial Planning	3	8
14	Settlement	3	13
15	E-commerce / OMS	11	19
16	HR & Payroll	5	22
17	Marketing	8	10
18	AI Content Engine	8	11
19	SEO, AEO & AIO	3	6
20	Projects & Collaboration	7	9
21	Dashboard & BI	5	9
22	AI Assistant, Agents & Automation	5	15

22 modules, 113 apps, 285 rules of which 86 are enforced by a named test. Every one of these figures is counted from the source at the moment this page was written; none was typed.

8.3 · The data model

151 tables, in build-phase order so the first phase can be run without reading the rest. Every business table carries a company, row-level security, FORCE, and a grant — all four, because three of them without the grant is a table nobody can read and three without the policy is a table everybody can.

Group	Tables	How they landed
E.1 · Quality & Compliance (Module 09)	4	3 new, 1 extending a table that already existed
E.2 · SEO, AEO & AIO (Module 19)	5	all new
E.3 · Projects & Collaboration — task coordination (Module 20)	4	all new
E.4 · Design & Sampling (Module 02)	5	4 new, 1 extending a table that already existed
E.5 · Partial-module closures	25	21 new, 4 extending a table that already existed

43 tables specified, 37 added and 6 folded into tables that already did the job. A duplicate table is worse than a missing one — two places to write a certificate, and two answers to how many expire this quarter — so each of the 6 names the columns it brought and a test checks them against a running database.

8.4 · What a business changes for itself

A tenant changes, without a developer	Who	When it takes effect
Somebody joins — a worker, a supervisor, an office staff member, a contractor	Admin, or an HR role	They exist from their start date and can be assigned work the same minute.
Somebody leaves, with or without notice	Admin, or an HR role	Marked as left from a date. Their sign-in stops, and no new work is assigned to them.

Their record is kept, not deleted. |

| A replacement starts immediately, in the same position | Admin, or an HR role | Added with their own start date and given the position. Work in progress is reassigned to them from that date. No waiting, no release, no developer. |

| A pay rate, a piece rate or a salary changes | Admin, or an HR role | Applies from the date you set — which may be today, a future date, or a past one you are correcting. |

| What somebody is allowed to see and do | Admin | Takes effect on their next action. Screens they may no longer open stop opening. |

| A new company is opened, or an existing one is closed | Admin | It exists immediately with its own name, trading name, code and document numbering. The group view includes it from that date. |

| A new way of selling is added — a marketplace, a shop, a counter, an export desk | Admin | Orders can arrive through it the same day. It appears in every report that breaks figures down by channel. |

| A godown, a shop, a unit or a stock point is added, renamed or closed | Admin | Stock can move to and from it immediately. |

| Which parts of the system this business uses at all | Admin | Turned on, a module appears in the menu with its screens ready. Turned off, it disappears from the menu. A steel plant, a clothing brand and a single creator each end up with a different system built from identical code. |

| What the system calls things | Admin | Every screen, every report and every document changes wording at once. One business says order, another says job, another says matter, consignment, batch or booking — the record underneath is identical. |

| Extra information you want to record that nobody else needs | Admin | Added to the screen immediately, with the type you choose — text, number, date, a list to pick from, a yes or no. Reportable from the moment it exists. |

| The steps your work moves through | Admin | Add a stage, rename one, reorder them or remove one. New work follows the new list from that moment. |

| The layout and numbering of invoices, statements, labels and reports | Admin | The next document uses the new layout or the new numbering. |

| Turning a discretionary rule on or off | Admin | Applies to the next transaction. One business requires an approval below a price floor; another does not — same software, different setting. |

| Who has to approve what, and above which amount | Admin | The next request follows the new path. |

| Tax rates and the categories they attach to | Admin, or an accounts role | Applies from its effective date, which for tax is set by law rather than by you. |

| Which outside service is used for messages, payments, delivery or artificial intelligence | Admin | The next message, payment or shipment goes through the new one. |

| The most the system may spend on paid outside services | Admin | Enforced immediately. Over the ceiling, the paid option is refused and the work completes on one that costs nothing. |

And what nobody changes — the half that makes the half above safe.

Fixed	Why it cannot be switched off
The audit trail	Who changed what, and when. A system where this can be switched off cannot be used to answer a dispute, so it cannot be switched off.
Every record naming the company it belongs to	Without it, figures from two companies merge and no report can be trusted again.
One business being unable to read another’s records	This is not a preference. It is the promise that makes a shared platform usable at all.
Money kept as exact whole units	The alternative loses fractions of a rupee in ways nobody can trace afterwards.
Deleting nothing — records are ended, never erased	An erased record changes a period that was already closed, filed and possibly audited.
Never asking for a marketplace, bank or account password	The system connects through proper keys that you can withdraw. A password would hand over an account you cannot take back.

18 things a business changes for itself, without a developer and without a release. And 6 nobody changes, which is the half that makes the first half safe: a business that could switch off its own audit trail could switch off the record of having done so.

Part 9 · What is real, and what is designed

This matters more than usual, because this document is written to be built FROM. Overstating what exists would mean whoever builds it is told to skip something that is not there.

Piece	State	How you can check
The data model — 151 tables	**Runs**	Loaded into a real PostgreSQL by a test that opens with a control which must leak before anything else is believed
Company and tenant isolation	**Runs**	Cross-company read and cross-company write both refused, asked of the database rather than asserted about the file
The payroll and production engine	**Runs**	Its own self-tests, covering the dated logs, the pay bases, set completion and the workbook that recalculates
The rulebook — 285 rules	**86 enforced**	An enforced rule must name a file and a test that exist, or the build fails
The 113 apps	**Designed**	A working subset exists as prototypes; the full set is what this document specifies
The 10 industry packs	**Run as data**	Gated by their own test, including the rule that a new trade is refused the same things the first ones are

19 capabilities, free-first. Every one starts on something that costs nothing, and a paid choice has to name both the free option it replaced and the specific trigger that justifies the money.

Part 10 · Every technical word on this page, in plain language

One glossary, shared by every document in this set. An agent building from this page should never have to guess what a word means, and a reader should never have to look one up somewhere else.

Word	What it means	The everyday version
platform	One piece of software that many separate businesses use at the same time, each seeing only its own information.	*Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.*
tenant	One business using the platform. Its people, its data and its settings are its own.	*Us building mein ek office. Aapka office, aapka saamaan, aapka taala.*
module	One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.	*Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.*
industry pack	A settings file that teaches the system your trade — what you call things, the stages your work moves through, the documents you issue.	*Ek hi machine, alag-alag saancha. Saancha badal do, wahi machine doosri cheez banane lagti hai.*
database	Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.	*Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.*
table	One kind of information inside the database — all your customers in one, all your orders in another.	*Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.*
row	One single record — one customer, one order, one payment.	*Register mein ek line. Ek line matlab ek entry.*
schema	The written plan of what information the system keeps and how the pieces connect.	*Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.*
row-level security	A lock inside the database itself, so one business physically cannot read another business's records — even if the software above it has a bug.	*Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.*
migration	A recorded change to the shape of the database, so every copy of the system can be updated the same way, in the same order.	*Naksha badla toh likh ke rakha — taaki har site pe wahi badlav, usi tarike se ho.*
backup	A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work.	*Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.*
integer paise	Money stored as a whole number of paise instead of a decimal, so amounts are exact and rounding can never quietly lose a rupee.	*Paisa hamesha poore paise mein ginte hain, aadha-adhoora kabhi nahin — isliye hisaab kabhi ek rupya idhar-udhar nahin hota.*
effective date	The date a change starts applying from. Records made before it keep the old value; records after it use the new one.	*Naya rate 1 tarikh se lagu. Purane mahine ka bill purane rate se hi banega — woh apne aap nahin badlega.*
audit trail	An automatic record of every change — what changed, who changed it, and when.	*Har entry ke saath naam aur time apne aap likha jaata hai. Baad mein koi bole "maine nahin kiya", toh register bata deta hai.*

Word	What it means	The everyday version
backend	The part of the software you never see, which does the actual work — checks the rules, saves the records, calculates the totals.	*Hotel ka kitchen. Customer nahin dekhta, par khaana wahin banta hai.*
frontend	The part you see and click — the screens, the buttons, the forms.	*Hotel ka dining hall aur menu card. Jo aapke saamne hai.*
API	The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer.	*Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.*
interface	A written promise about what a part of the system does, without saying which product does it — so the product underneath can be swapped without anything above noticing.	*Bijli ka socket. Socket ka size fix hai; usme koi bhi company ka plug lag jaata hai.*
adapter	A small piece of code that translates between the system and one outside service, so the rest of the system never has to know which service is in use.	*Travel adapter. Andar ka appliance wahi, bas plug ko us desk ke socket ke hisaab se badal diya.*
storage	Where files are kept — photographs, invoices, scanned documents. Different from the database, which keeps information rather than files.	*Almari ke bagal wala godown. Register almari mein, par bade dabbe aur photo godown mein.*
cache	A small, fast copy of information that was just looked up, kept ready in case it is asked for again.	*Counter pe rakha hua sabse zyada bikne wala saamaan. Har baar godown tak jaana nahin padta.*
queue	A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.	*Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.*
job	One piece of work taken off the queue and done in the background.	*Line mein se uthayi gayi ek parchi, ab uska kaam ho raha hai.*
search index	A prepared list that makes finding things fast, the way the index at the back of a book beats reading every page.	*Kitaab ke peeche wali index. Poori kitaab padhne ki zaroorat nahin, seedha page number mil jaata hai.*
environment	A separate running copy of the system — one for trying things, one that customers actually use.	*Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.*
deployment	Putting a new version of the software in place so people start using it.	*Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.*
continuous integration	A robot that checks every change automatically, before anyone can put it live.	*Quality-check wala banda gate pe khada. Har maal nikalne se pehle usse guzarta hai.*
rollback	Putting the previous working version back, quickly, when a new one turns out to be wrong.	*Naya taala kharab nikla toh purana taala wapas laga do — do minute ka kaam.*

Word	What it means	The everyday version
observability	Being able to see what the system is doing and what went wrong, without guessing.	*Dukaan mein CCTV aur register. Kuch gadbad ho toh dekh sakte ho ki hua kya, andaaza nahin lagana padta.*
uptime	How much of the time the system is actually working and reachable.	*Dukaan mahine mein kitne din khuli rahi. Band rahi toh customer wapas chala gaya.*
model	The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question.	*Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.*
provider	A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery.	*Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.*
fallback	The next option the system automatically moves to when the first one fails or is unavailable.	*Bijli gayi toh inverter. Inverter gaya toh mombatti. Andhera kabhi nahin hota.*
spend ceiling	A maximum amount the system is allowed to spend on paid services, after which it refuses to spend more instead of warning you.	*Jeb mein utne hi paise leke nikle jitna kharch karna hai. Khatam matlab khatam — udhaar nahin.*
circuit breaker	A switch that takes a repeatedly failing service out of use for a while, instead of retrying it endlessly and slowing everything down.	*Ghar ka MCB. Baar-baar fault aa raha hai toh woh line hi kaat deta hai, poora ghar band nahin hota.*
role	What a person is allowed to see and do — a manager sees more than a counter staff member.	*Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.*
permission	One specific thing a role is allowed to do, like approving a discount or viewing salaries.	*Guchhe ki ek chaabi. Ek chaabi ek darwaza.*
authentication	Proving you are who you say you are, usually by signing in.	*Gate pe pehchaan dikhana. "Main kaun hoon" wala sawaal.*
encryption	Scrambling information so that even somebody who steals the file cannot read it.	*Apni hi code-bhasha mein likhna. Chori bhi ho jaaye toh padha nahin jaata.*

Generated by `brand/delivery/website/mkarchitect.js` from `brand/site/architect.js` and the canonical sources it names. 2026-08-28. Nothing here was retyped: regenerate rather than editing this file.